Concurrent Data Structures in Java: Lazy and Non-Blocking Synchronization

HPC Programming Assignment 2
Tejaansh (SE23UCSE174) · Pallavi (SE23UCSE135)

1. Overview

Seven Java implementations were built and benchmarked: a lazy and a lock-free doubly linked list, a lazy lock-based stack and a Treiber stack, a two-lock queue and a Michael-Scott queue, and an elimination backoff stack built on the Treiber stack. All of them implement one small interface (`read/insert/delete`) so a single harness (`Bench.java`) drives every experiment. `Check.java` is a multi-threaded correctness test: it checks set semantics, that prev links mirror next links, and that every pushed or enqueued value is removed exactly once.

2. Implementations

2.1 Lazy doubly linked list (`LazyDList`)

The list is sorted, with sentinels `head` (MIN) and `tail` (MAX). Each node has `key`, volatile `next`, `prev` and `marked`, plus a `ReentrantLock`. A thread traverses `next` links without locking, locks `pred` and then `curr`, and validates `!pred.marked && !curr.marked && pred.next == curr`. If validation fails, it retries. Insert sets `node.next=curr, node.prev=pred`, then `pred.next=node`, then `curr.prev=node`. Delete first marks `curr` (logical deletion), then unlinks it (physical deletion) with `pred.next=succ; succ.prev=pred`. The `prev` links stay consistent because a node's `prev` is only written by a thread that holds the lock of that node's predecessor. While a thread holds `curr`'s lock, nobody else can change `succ.prev`. `contains()` is wait-free and takes no locks.

2.2 Non-blocking doubly linked list (`LockFreeDList`)

This list uses the Harris-Michael design. Each `next` field is an `AtomicMarkableReference`, so the successor and the deleted mark change together in one CAS. Delete marks `curr.next` with `compareAndSet(succ, succ, false, true)` (logical deletion), then tries to unlink with a CAS on `pred.next` (physical deletion). `attemptMark` is deliberately not used: it returns true when the node is already marked, which would let two deleters both report success. `find()` helps other threads. Any marked node it passes is unlinked with a CAS. If that CAS fails, the traversal restarts from `head`, and every failed insert or delete CAS also retries. The `prev` pointers are updated after each successful link or unlink, but correctness never depends on them; the `next` chain is the authoritative structure. This avoids the multi-word CAS that a fully lock-free, symmetric doubly linked list would need. No locks appear on any update path.

2.3 Lazy lock-based stack (`LockStack`)

Push and pop read `top` without the lock and prepare the new node (`node.next = observed`). They then take the lock, check `top == observed`, and only then publish. If the check fails, they retry. A pop that reads an empty stack returns without locking. `peek()` is a single volatile read.

2.4 Treiber stack (`TreiberStack`)

`top` is an `AtomicReference`. Push links the new node to the observed top and tries `CAS(top, oldTop, node)`. Pop tries `CAS(top, oldTop, oldTop.next)`. Both retry when the CAS fails. Java's garbage collector prevents ABA, because a node cannot be reused while any thread still references it.

2.5 Two-lock queue (`LockQueue`)

The queue starts with a sentinel node and uses separate `headLock` and `tailLock`, so one enqueuer and one dequeuer can run at the same time. Enqueue appends under `tailLock`. Dequeue reads `head` and `head.next` without the lock. It returns empty if `head.next` is null; otherwise it takes `headLock`, checks that `head` is unchanged, and advances `head`. If the check fails, it retries.

2.6 Michael-Scott queue (`MSQueue`)

`head`, `tail` and every `next` field are `AtomicReference`s. Enqueue CASes `last.next` from null to the new node, then tries to swing `tail`. Any thread that finds `tail` lagging behind (`tail.next != null`) helps by advancing it. Dequeue CASes `head` forward. Peek reads `head.next` and retries if `head` changed meanwhile.

GC nepotism (both queues). In a first run of the experiments, queue runs overshot their deadline by up to 0.8 s, and throughput fell steadily over the life of each JVM. GC logs showed young-generation pauses of 125–160 ms and a live heap growing from 95 MB to 290 MB. The cause is that a dequeued sentinel promoted to the old generation still points to younger nodes and keeps them alive, so every young collection has to copy the entire queue history. Both queues therefore self-link the dead sentinel after a successful dequeue (`h.next = h`), as `java.util.concurrent.ConcurrentLinkedQueue` does. This is safe because any thread still holding the old node fails its `head`/`tail` check and retries. After the fix, the pauses disappeared and all queue experiments were rerun.

2.7 Elimination backoff stack (`EliminationStack`)

This class extends `TreiberStack` and reuses its single-attempt `tryPush`/`tryPop`. When a CAS on `top` fails, the thread backs off into an elimination array of 8 lock-free exchangers and picks a slot at random with `ThreadLocalRandom`. Each exchanger is an `AtomicStampedReference` with three states: EMPTY, WAITING and BUSY. The first thread CASes EMPTY to WAITING and spins. A partner CASes WAITING to BUSY and takes the waiting thread's item. Pushes offer their value and pops offer `null`, so a push that meets a pop completes both operations without touching `top`. Two pushes or two pops that meet simply retry. The backoff policy is a 2 μs timeout per visit, after which the thread goes back to the central stack.

3. Linearization points

Structure	Operation	Linearization point
LazyDList	insert ok / fail	write `pred.next = node` / validation that found unmarked `curr.key == key`

	delete ok / fail	write `curr.marked = true` / validation that found `curr.key != key`
	contains true / false	read of `curr.marked == false` / read of first node with key ≥ target, or the moment the matching node was marked, whichever is later
LockFreeDList	insert ok / fail	successful CAS `pred.next: curr→node` / read in `find()` returning unmarked `curr` with equal key
	delete ok / fail	successful CAS marking `curr.next` / read in `find()` returning `curr.key != key`
	contains true / false	read of the mark bit of `curr.next` (unmarked) / as for the lazy list
LockStack	push / pop / empty pop / peek	write of `top` under the lock after validation / same / volatile read `top == null` / volatile read of `top`
TreiberStack	push / pop / empty pop / peek	successful CAS on `top` / successful CAS on `top` / read `top == null` / read of `top`
LockQueue	enqueue / dequeue / empty dequeue / peek	write `tail.next = node` / write `head = first` after validation / read `head.next == null` / read of `head.next` confirmed by an unchanged `head`
MSQueue	enqueue / dequeue / empty dequeue / peek	successful CAS `last.next: null→node` / successful CAS on `head` / read `first.next == null` while `head == tail` / read of `head.next` confirmed by an unchanged `head`
EliminationStack	push / pop via stack	as for the Treiber stack
	eliminated pair	the partner's successful CAS WAITING→BUSY; the push is ordered immediately before the pop

4. Experimental methodology

- **Grid:** 7 implementations × N ∈ {1,000, 10,000} × 3 workloads (0C-50I-50D, 50C-25I-25D, 100C-0I-0D) × threads {1,2,4,6,8,10,12,14,16} × 5 runs, for 1,890 timed runs in total.
- **Run duration (deviation from the brief):** each timed run lasts **2,000 ms** rather than the specified 10,000 ms, to fit the full grid into the available time on a single 4-core machine. The duration is a command-line parameter: `scripts/run_all.sh 10000 5` reruns the whole grid at the specified 10 s. Because throughput is normalised by the measured elapsed time and every run has the same duration, throughput and speedup are computed exactly as specified.
- **Prefill:** before timing, N/2 distinct keys (a shuffled subset of [0, N)) are inserted, pushed or enqueued. List keys are drawn from [0, N), so the 50/50 insert/delete mix keeps the list about 50% full.
- **Workload generation:** before timing, each thread gets a pre-generated array of 2^{18} (op, key) entries drawn with the workload's percentages. Threads cycle through it, so no random numbers are generated inside the timed loop.
- **Timing:** each run uses a fresh structure. Threads are created and started first and then wait on a `CountDownLatch`. The main thread takes `start = System.nanoTime()`, publishes `deadline = start + duration`, and opens the latch. Each worker runs `while (System.nanoTime() < deadline)` and executes 64 operations per check, counting completed operations. The batching matters. On macOS, HotSpot's `System.nanoTime()` performs a CAS on one global word to guarantee monotonic time. We measured this capping all threads together at about 30 M calls/s at 1 thread, falling to 26 M at 4 threads. With one clock call per operation, every fast structure (stacks, queues, peek) saturated near 29 MOPS regardless of thread count, so the benchmark was measuring the clock rather than the data structure. Checking every 64 operations removes

that artefact and overshoots the deadline by well under 1 ms of a 2,000 ms run (the measured elapsed time is used in any case). After all threads join, `end = System.nanoTime()` is taken. Only `System.nanoTime()` is used for timing.

- **Metrics:** throughput (MOPS) = ops / (elapsed_s × 10^6). P_{avg} is the mean of the 5 runs, and speedup(p) = $P_{avg}(p)$ / $P_{avg}(1)$ for the same implementation, N and workload.
- **JVM:** Java 17 HotSpot with `-Xms2g -Xmx2g`. One JVM is started per (implementation, N, workload), and a discarded 1-second warm-up run lets the JIT compile the hot paths first. JVM startup, prefill, generation and output are all outside the timed interval.
- **Hardware:** a 4-core Intel Mac (macOS 13) with no SMT (`hw.ncpu = 4`). Every configuration above 4 threads therefore oversubscribes the CPU, and the OS scheduler time-slices threads. This shapes the results at 6–16 threads.

To reproduce: run `scripts/run_all.sh`, which compiles, runs the grid and resumes where it stopped. Then run `python3 scripts/analyze.py`, `gnuplot scripts/plots.gp` and `python3 scripts/make_report.py`. The raw data for every run is in `results/raw/*.csv` and `results/all_runs.csv`. Averages and speedups are in `results/averages.csv`.

5. Results

N = 1000: average throughput in MOPS (speedup in parentheses)

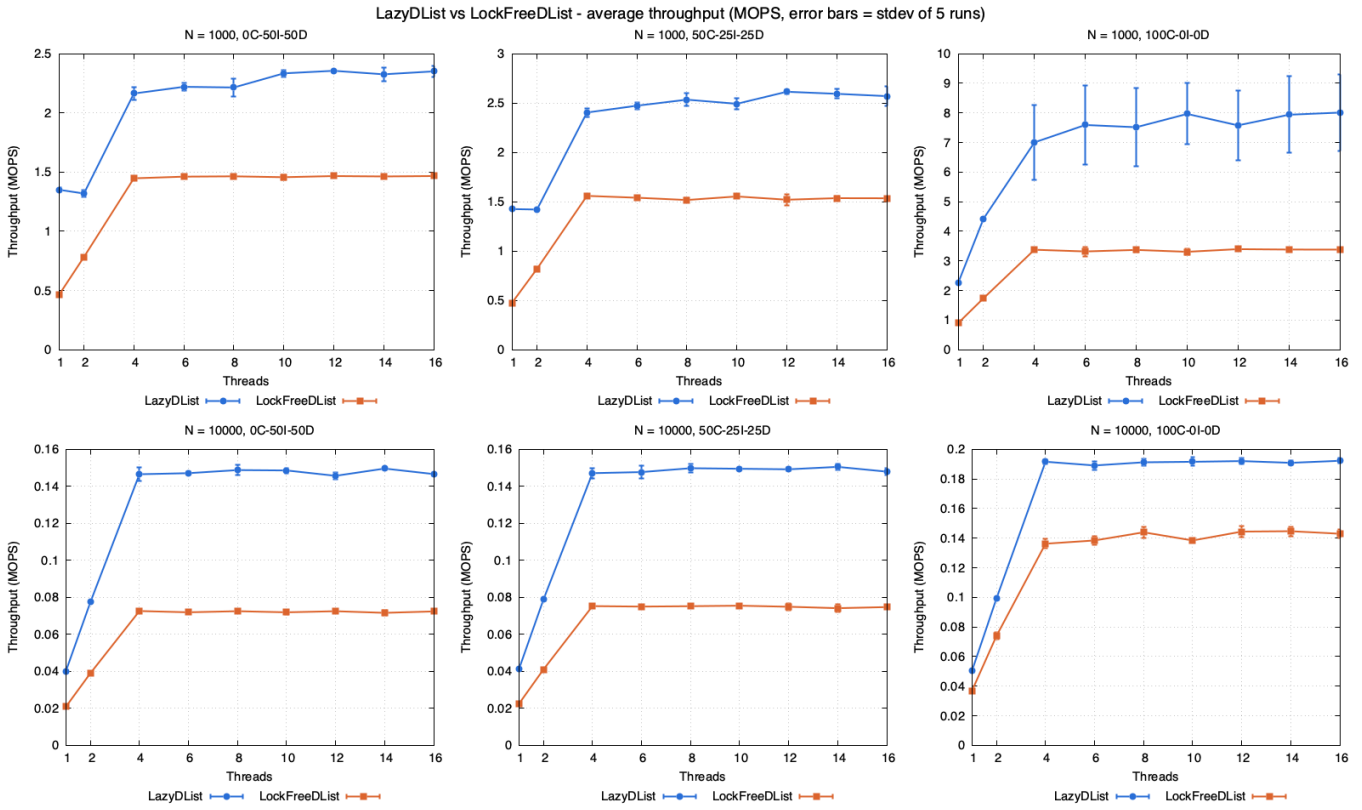
Impl	Workload	1	2	4	6	8	10	12	14	16
LazyDList	0C-50I-50D	1.35 (1.00)	1.32 (0.98)	2.16 (1.60)	2.22 (1.64)	2.21 (1.64)	2.33 (1.73)	2.36 (1.74)	2.33 (1.72)	2.35 (1.74)
LazyDList	50C-25I-25D	1.42 (1.00)	1.42 (1.00)	2.40 (1.69)	2.47 (1.74)	2.53 (1.78)	2.49 (1.75)	2.61 (1.84)	2.59 (1.82)	2.57 (1.80)
LazyDList	100C-0I-0D	2.25 (1.00)	4.42 (1.97)	7.00 (3.12)	7.59 (3.38)	7.52 (3.34)	7.96 (3.54)	7.58 (3.37)	7.94 (3.53)	8.01 (3.56)
LockFreeDList	0C-50I-50D	0.46 (1.00)	0.78 (1.69)	1.45 (3.14)	1.46 (3.17)	1.46 (3.17)	1.46 (3.16)	1.47 (3.18)	1.46 (3.17)	1.47 (3.18)
LockFreeDList	50C-25I-25D	0.47 (1.00)	0.82 (1.72)	1.56 (3.29)	1.54 (3.25)	1.52 (3.20)	1.55 (3.27)	1.52 (3.20)	1.54 (3.24)	1.53 (3.23)
LockFreeDList	100C-0I-0D	0.89 (1.00)	1.74 (1.95)	3.37 (3.78)	3.32 (3.71)	3.37 (3.77)	3.30 (3.70)	3.39 (3.80)	3.38 (3.79)	3.38 (3.78)
LockStack	0C-50I-50D	30.66 (1.00)	8.41 (0.27)	19.45 (0.63)	19.46 (0.63)	19.04 (0.62)	19.45 (0.63)	19.29 (0.63)	19.57 (0.64)	19.17 (0.63)
LockStack	50C-25I-25D	56.77 (1.00)	15.34 (0.27)	35.89 (0.63)	35.75 (0.63)	35.34 (0.62)	35.50 (0.63)	35.13 (0.62)	34.17 (0.60)	33.89 (0.60)
LockStack	100C-0I-0D	700.97 (1.00)	949.53 (1.35)	1437.59 (2.05)	1441.91 (2.06)	1443.80 (2.06)	1442.97 (2.06)	1445.68 (2.06)	1438.74 (2.05)	1444.61 (2.06)
TreiberStack	0C-50I-50D	51.13 (1.00)	14.33 (0.28)	9.57 (0.19)	9.37 (0.18)	9.49 (0.19)	9.50 (0.19)	9.46 (0.18)	10.34 (0.20)	9.57 (0.19)
TreiberStack	50C-25I-25D	77.25 (1.00)	28.06 (0.36)	18.99 (0.25)	18.33 (0.24)	18.29 (0.24)	18.54 (0.24)	19.34 (0.25)	18.04 (0.23)	18.82 (0.24)
TreiberStack	100C-0I-0D	591.53 (1.00)	849.38 (1.44)	1561.88 (2.64)	1546.47 (2.61)	1529.43 (2.59)	1527.78 (2.58)	1520.84 (2.57)	1538.65 (2.60)	1539.05 (2.60)
EliminationStack	0C-50I-50D	57.01 (1.00)	48.46 (0.85)	35.77 (0.63)	35.10 (0.62)	38.47 (0.67)	37.57 (0.66)	41.47 (0.73)	34.45 (0.60)	35.83 (0.63)
EliminationStack	50C-25I-25D	68.38 (1.00)	61.83 (0.90)	48.51 (0.71)	50.10 (0.73)	50.16 (0.73)	49.10 (0.72)	50.00 (0.73)	51.76 (0.76)	48.48 (0.71)

EliminationStack	100C-0I-0D	602.03 (1.00)	848.41 (1.41)	1557.24 (2.59)	1529.20 (2.54)	1541.93 (2.56)	1540.75 (2.56)	1543.87 (2.56)	1524.54 (2.53)	1542.42 (2.56)
LockQueue	0C-50I-50D	29.72 (1.00)	10.43 (0.35)	17.80 (0.60)	17.60 (0.59)	17.85 (0.60)	17.96 (0.60)	17.20 (0.58)	16.89 (0.57)	18.01 (0.61)
LockQueue	50C-25I-25D	53.60 (1.00)	20.07 (0.37)	33.35 (0.62)	32.98 (0.62)	32.43 (0.61)	32.37 (0.60)	32.69 (0.61)	32.92 (0.61)	32.08 (0.60)
LockQueue	100C-0I-0D	473.81 (1.00)	734.48 (1.55)	1412.89 (2.98)	1410.67 (2.98)	1411.21 (2.98)	1408.08 (2.97)	1407.71 (2.97)	1411.26 (2.98)	1410.95 (2.98)
MSQueue	0C-50I-50D	38.56 (1.00)	12.64 (0.33)	8.20 (0.21)	8.54 (0.22)	8.14 (0.21)	8.03 (0.21)	9.31 (0.24)	9.00 (0.23)	9.74 (0.25)
MSQueue	50C-25I-25D	60.68 (1.00)	24.59 (0.41)	18.09 (0.30)	17.54 (0.29)	17.47 (0.29)	16.57 (0.27)	17.01 (0.28)	18.53 (0.31)	17.64 (0.29)
MSQueue	100C-0I-0D	355.95 (1.00)	559.38 (1.57)	1061.44 (2.98)	1063.39 (2.99)	1046.44 (2.94)	1064.30 (2.99)	1064.90 (2.99)	1060.89 (2.98)	1056.02 (2.97)

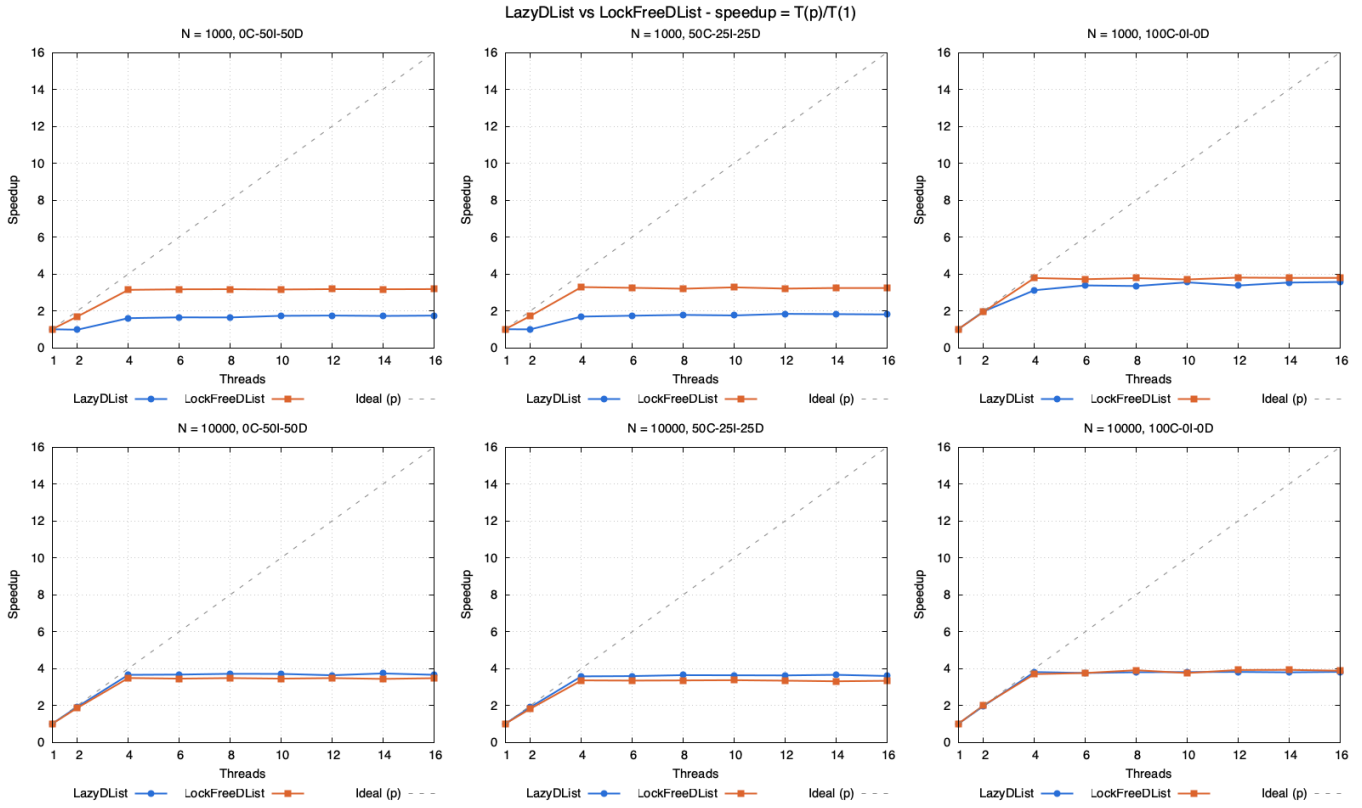
N = 10000: average throughput in MOPS (speedup in parentheses)

Impl	Workload	1	2	4	6	8	10	12	14	16
LazyDList	0C-50I-50D	0.04 (1.00)	0.08 (1.93)	0.15 (3.66)	0.15 (3.68)	0.15 (3.72)	0.15 (3.71)	0.15 (3.64)	0.15 (3.74)	0.15 (3.66)
LazyDList	50C-25I-25D	0.04 (1.00)	0.08 (1.92)	0.15 (3.58)	0.15 (3.59)	0.15 (3.64)	0.15 (3.64)	0.15 (3.63)	0.15 (3.66)	0.15 (3.60)
LazyDList	100C-0I-0D	0.05 (1.00)	0.10 (1.97)	0.19 (3.81)	0.19 (3.76)	0.19 (3.80)	0.19 (3.81)	0.19 (3.82)	0.19 (3.80)	0.19 (3.82)
LockFreeDList	0C-50I-50D	0.02 (1.00)	0.04 (1.87)	0.07 (3.49)	0.07 (3.46)	0.07 (3.48)	0.07 (3.45)	0.07 (3.48)	0.07 (3.44)	0.07 (3.48)
LockFreeDList	50C-25I-25D	0.02 (1.00)	0.04 (1.83)	0.08 (3.36)	0.07 (3.35)	0.08 (3.36)	0.08 (3.37)	0.07 (3.35)	0.07 (3.31)	0.07 (3.34)
LockFreeDList	100C-0I-0D	0.04 (1.00)	0.07 (2.01)	0.14 (3.70)	0.14 (3.76)	0.14 (3.91)	0.14 (3.76)	0.14 (3.92)	0.14 (3.93)	0.14 (3.88)
LockStack	0C-50I-50D	30.70 (1.00)	9.28 (0.30)	19.63 (0.64)	19.59 (0.64)	19.96 (0.65)	19.83 (0.65)	20.02 (0.65)	19.37 (0.63)	18.84 (0.61)
LockStack	50C-25I-25D	57.87 (1.00)	17.78 (0.31)	36.41 (0.63)	35.50 (0.61)	35.81 (0.62)	35.13 (0.61)	34.76 (0.60)	34.05 (0.59)	34.76 (0.60)
LockStack	100C-0I-0D	714.61 (1.00)	965.93 (1.35)	1450.52 (2.03)	1454.24 (2.04)	1451.41 (2.03)	1453.35 (2.03)	1444.52 (2.02)	1451.49 (2.03)	1446.73 (2.02)
TreiberStack	0C-50I-50D	53.75 (1.00)	13.94 (0.26)	9.46 (0.18)	9.06 (0.17)	9.36 (0.17)	9.21 (0.17)	9.56 (0.18)	9.66 (0.18)	9.45 (0.18)
TreiberStack	50C-25I-25D	75.72 (1.00)	28.22 (0.37)	17.45 (0.23)	17.90 (0.24)	18.29 (0.24)	18.04 (0.24)	18.67 (0.25)	18.56 (0.25)	19.56 (0.26)
TreiberStack	100C-0I-0D	601.82 (1.00)	845.24 (1.40)	1567.42 (2.60)	1564.85 (2.60)	1539.98 (2.56)	1540.35 (2.56)	1548.11 (2.57)	1556.87 (2.59)	1546.98 (2.57)
EliminationStack	0C-50I-50D	55.31 (1.00)	49.44 (0.89)	36.23 (0.66)	35.78 (0.65)	38.60 (0.70)	39.67 (0.72)	39.49 (0.71)	38.32 (0.69)	36.49 (0.66)
EliminationStack	50C-25I-25D	81.51 (1.00)	71.72 (0.88)	54.23 (0.67)	54.57 (0.67)	57.18 (0.70)	55.37 (0.68)	58.38 (0.72)	58.45 (0.72)	58.06 (0.71)
EliminationStack	100C-0I-0D	598.74 (1.00)	835.95 (1.40)	1576.93 (2.63)	1562.67 (2.61)	1547.71 (2.58)	1557.53 (2.60)	1556.90 (2.60)	1549.35 (2.59)	1549.24 (2.59)
LockQueue	0C-50I-50D	29.68 (1.00)	11.62 (0.39)	17.98 (0.61)	17.86 (0.60)	18.07 (0.61)	18.04 (0.61)	17.76 (0.60)	16.82 (0.57)	17.54 (0.59)
LockQueue	50C-25I-25D	55.10 (1.00)	20.64 (0.37)	33.89 (0.61)	33.35 (0.61)	33.42 (0.61)	32.68 (0.59)	33.41 (0.61)	33.04 (0.60)	33.48 (0.61)
LockQueue	100C-0I-0D	478.92 (1.00)	727.77 (1.52)	1389.67 (2.90)	1389.47 (2.90)	1391.86 (2.91)	1392.48 (2.91)	1375.35 (2.87)	1359.91 (2.84)	1390.77 (2.90)
MSQueue	0C-50I-50D	37.52 (1.00)	12.22 (0.33)	7.79 (0.21)	8.34 (0.22)	7.85 (0.21)	7.83 (0.21)	8.96 (0.24)	9.84 (0.26)	8.05 (0.21)

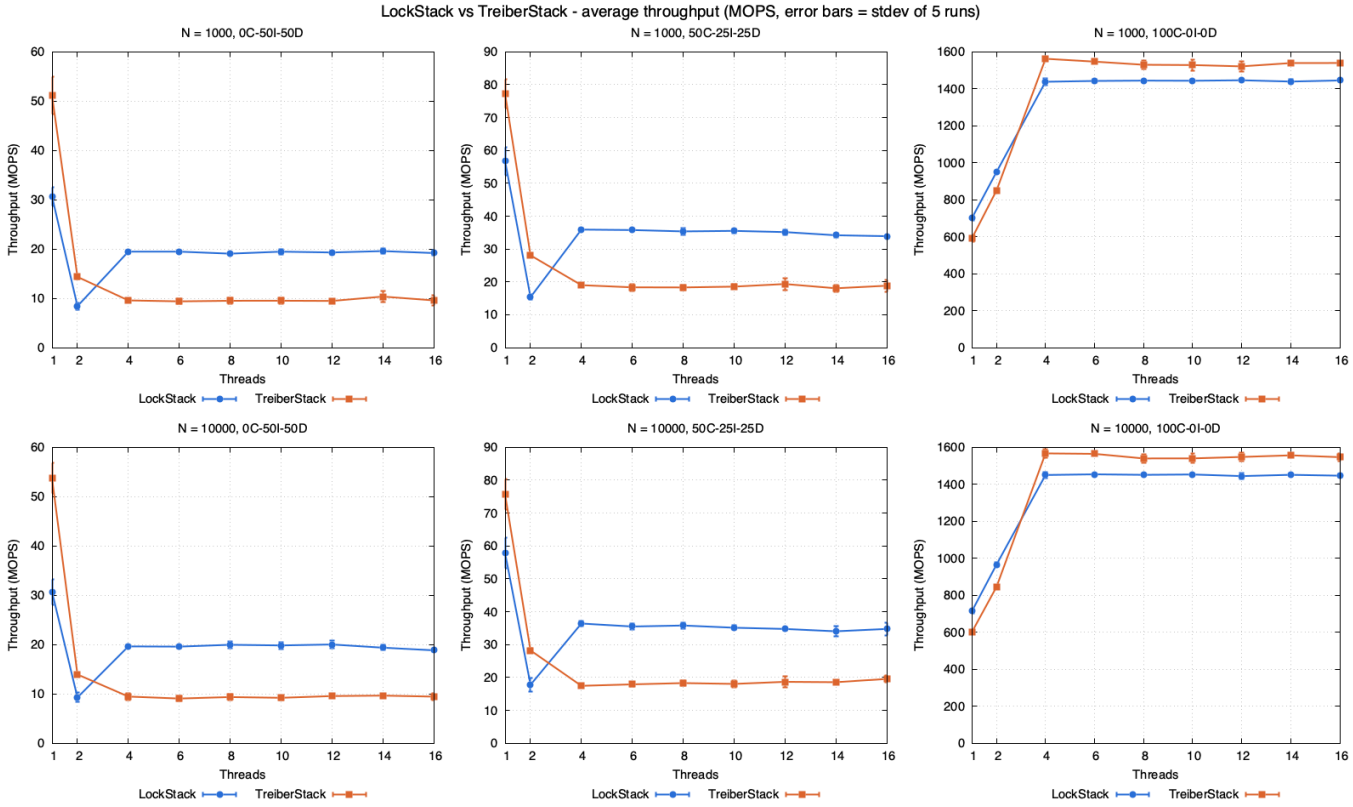
MSQueue	50C-25I-25D	60.92 (1.00)	24.18 (0.40)	18.01 (0.30)	16.04 (0.26)	17.64 (0.29)	16.34 (0.27)	17.54 (0.29)	17.56 (0.29)	18.92 (0.31)
MSQueue	100C-0I-0D	357.06 (1.00)	558.49 (1.56)	1071.83 (3.00)	1060.72 (2.97)	1066.87 (2.99)	1065.10 (2.98)	1065.78 (2.98)	1061.74 (2.97)	1067.49 (2.99)



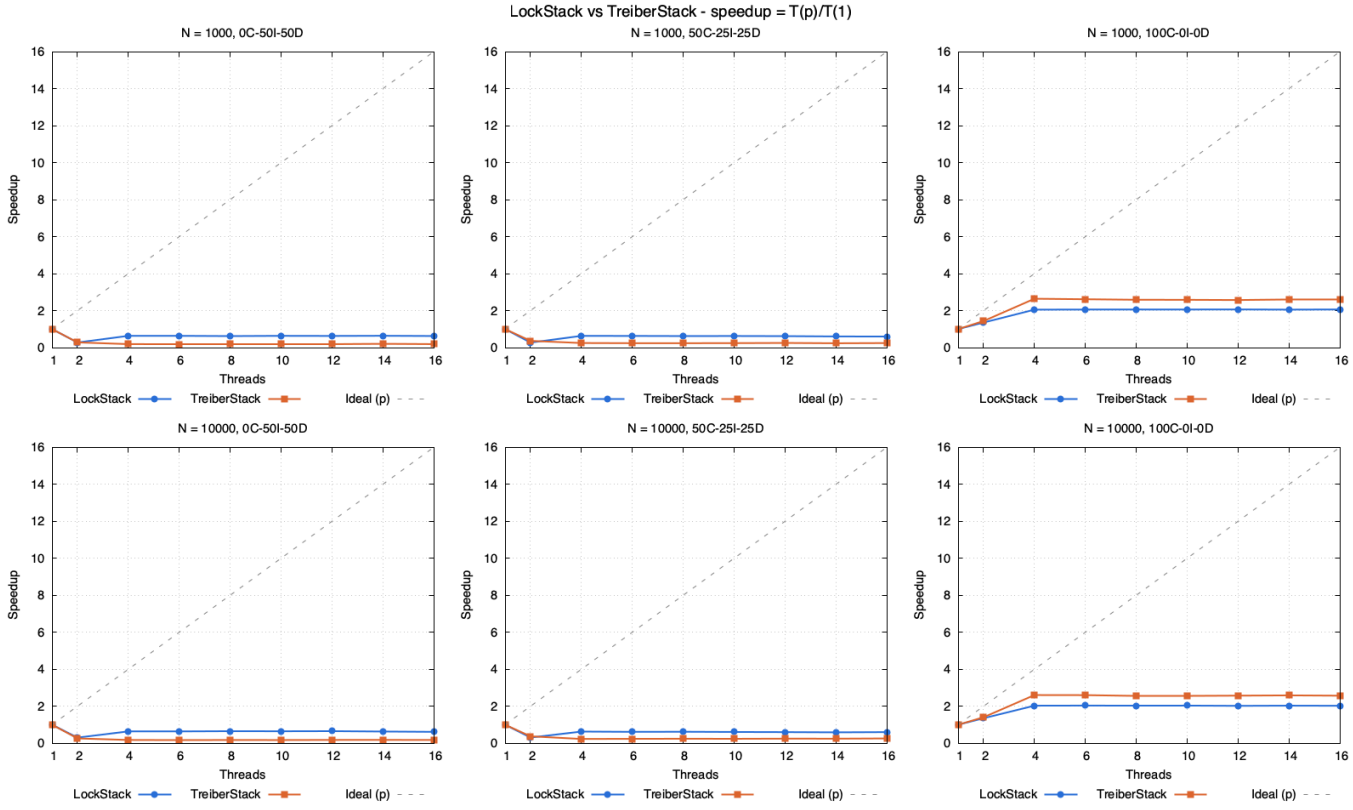
Doubly linked list: Lazy vs Non-Blocking - throughput



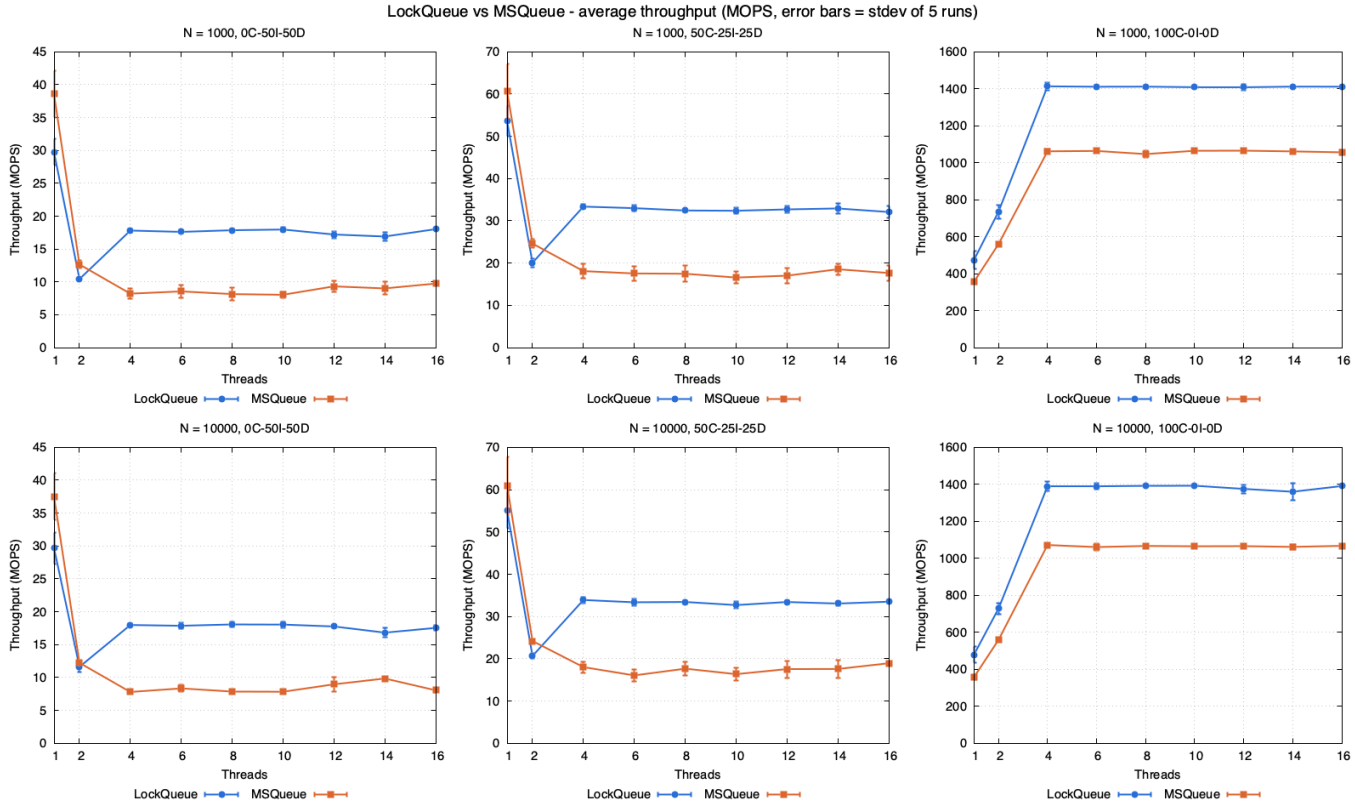
Doubly linked list: Lazy vs Non-Blocking - speedup



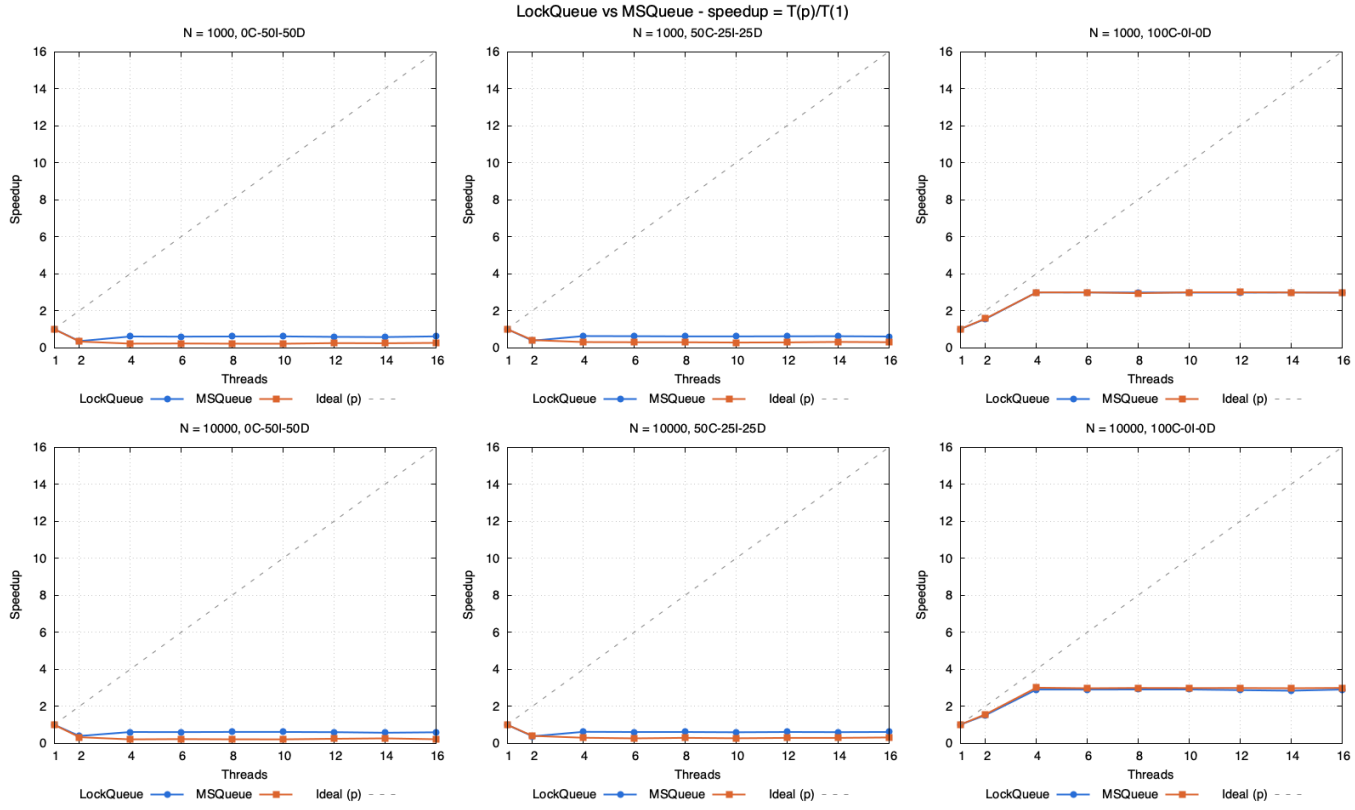
Stack: Lazy lock-based vs Treiber - throughput



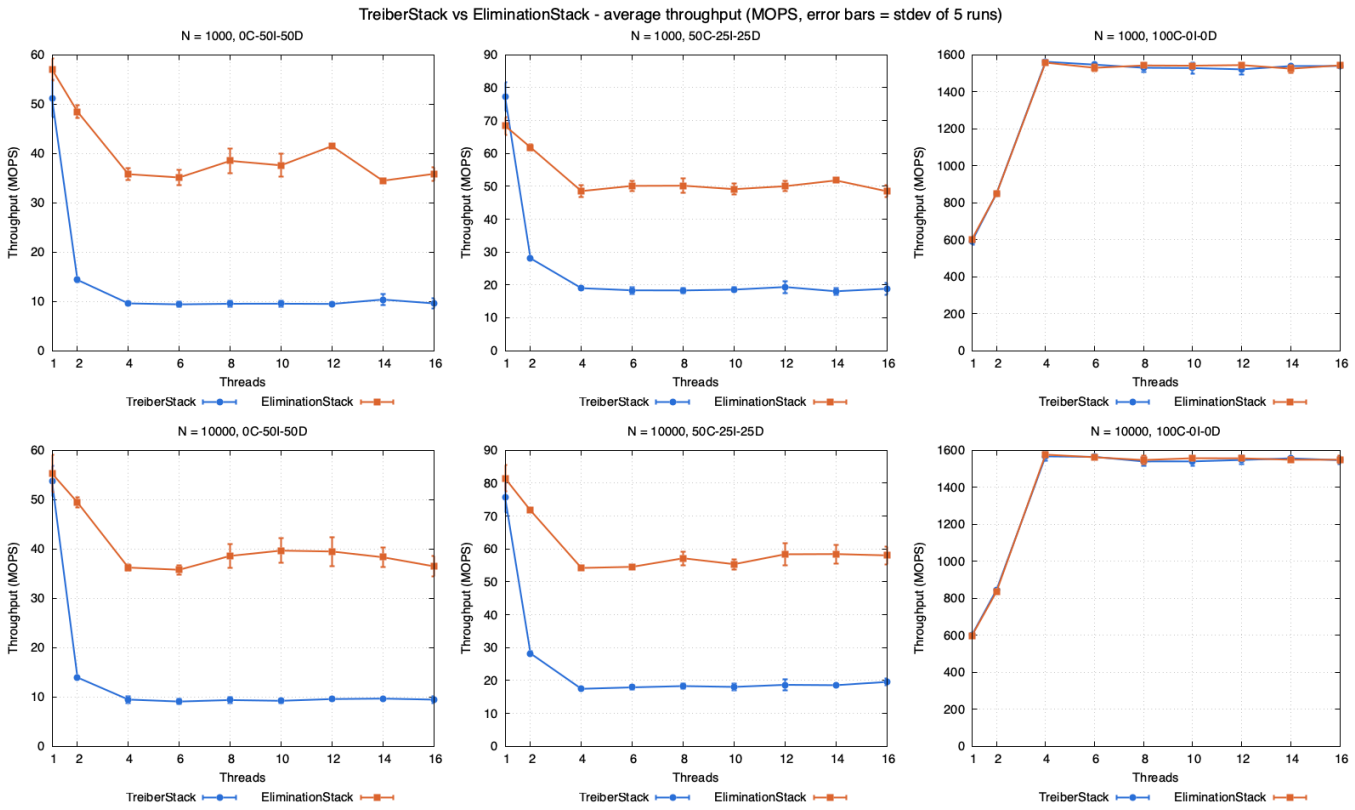
Stack: Lazy lock-based vs Treiber - speedup



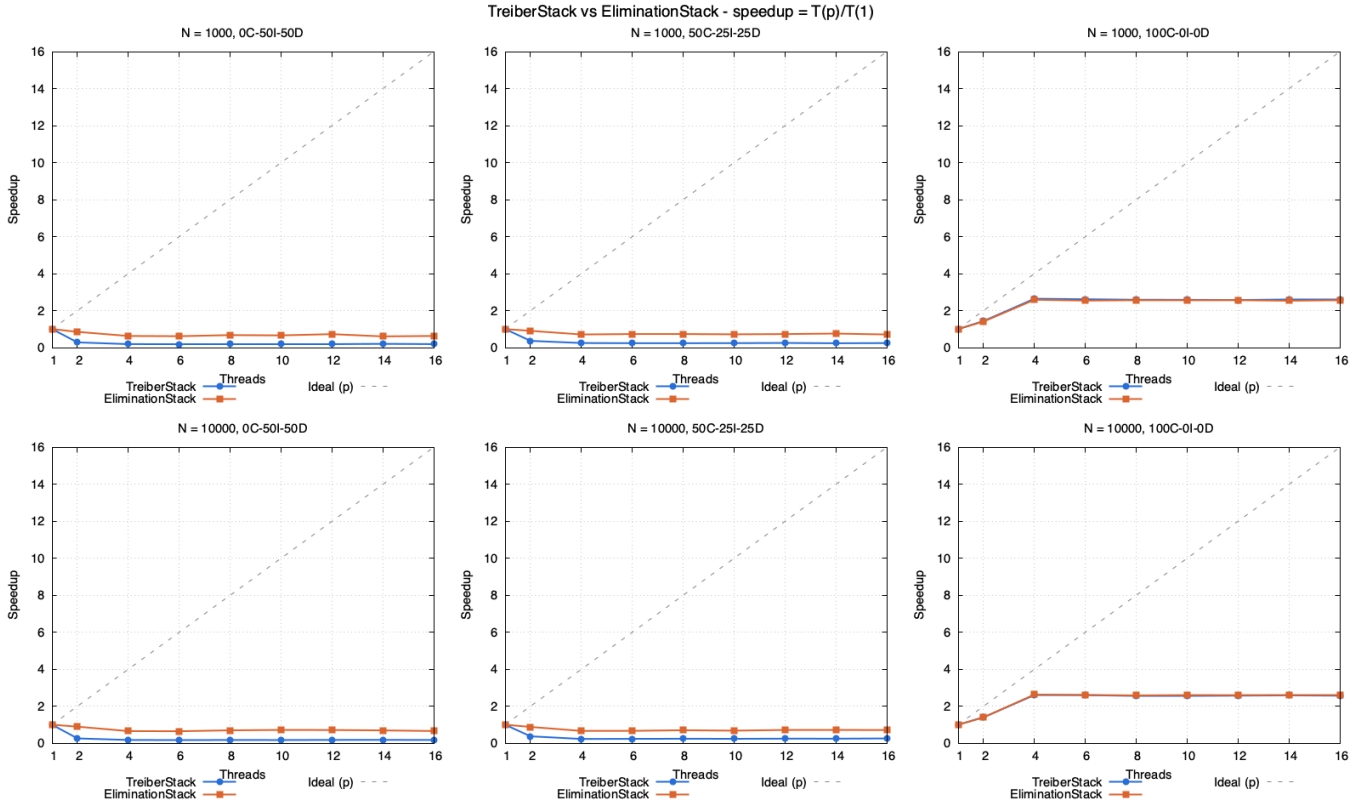
Queue: two-lock vs Michael-Scott - throughput



Queue: two-lock vs Michael-Scott - speedup



Treiber vs Elimination Backoff stack - throughput



Treiber vs Elimination Backoff stack - speedup

6. Analysis

All figures quoted below are 5-run averages, in MOPS. The machine has **4 physical cores**, so ideal speedup is bounded by 4. Every configuration above 4 threads is oversubscribed and relies on OS time-slicing, which is why every curve is flat from 4 to 16 threads. Across all 42 configurations no run collapsed under oversubscription, and the coefficient of variation stayed at or below about 12% (one outlier: lazy list, N=1000, read-only, at 18%).

6.1 Doubly linked list: lazy vs non-blocking

- **Scalability comes from traversal parallelism.** A list operation spends almost all of its time walking about N/4 nodes, and walking is read-only, so threads only share cache lines in the Shared state. At N=10,000 both lists scale almost linearly up to the core count. At 4 threads the lazy list reaches 3.66× (update-only) and 3.81× (read-only), and the lock-free list reaches 3.49× and 3.70×. Conflicts on the same pair of nodes are rare in a list of about 5,000 nodes.
- **At N=1,000, lock contention limits the lazy list.** Update-heavy speedup is only 1.60× (0C-50I-50D) and 1.69× (50C-25I-25D) at 4 threads, and peaks around 1.8×. With only about 500 nodes, two threads frequently need overlapping `pred/curr` locks. A thread that fails validation must re-traverse, and the lock words and `marked` flags of hot nodes bounce between cores (cache-coherence traffic). At 2 threads the lazy list gains nothing (1.35 → 1.32 MOPS).
- **The lock-free list scales better at N=1,000**: 3.14× (0C-50I-50D) and 3.29× (50C-25I-25D) at 4 threads. A failed CAS costs only a local retry rather than a blocked thread, and no thread ever waits for a descheduled lock holder.
- **The lazy list has higher absolute throughput anyway.** At N=1,000 and 16 threads it reaches 2.35 MOPS against 1.47 for the lock-free list (update-only), and 8.0 against 3.4 MOPS (read-only). Each step of the lock-free traversal goes through an `AtomicMarkableReference`: node → reference object → immutable (ref, mark) pair → next node. That is about three dependent loads per hop instead of one, and every successful CAS allocates a new pair. The lazy list follows plain volatile fields. In other words, the non-blocking version wins on scalability and lock-freedom but pays a constant synchronization overhead per node visited.
- **The effect of N is algorithmic.** Throughput drops by roughly 15roughly 30–45×ndash;40roughly 15–40× from N=1,000 to N=10,000 (e.g. lazy list read-only, 4 threads: 7.0 → 0.19 MOPS), consistent with O(N) traversal and with the larger list no longer fitting in L1/L2 cache.

6.2 Stack: lazy lock-based vs Treiber

- **Update workloads do not scale for either stack. A stack has one hot spot (`top`) and serialises all updates.** In the update-only workload at N=1,000, the Treiber stack falls from 51.1 MOPS at 1 thread to 14.3 at 2 and 9.6 at 4 (speedup 0.19). Every push and pop CASes the same cache line. Under contention most CASes fail (**CAS failures and retries**), and each attempt forces the line to move between cores in exclusive state, so throughput is set by cache-line transfer latency rather than by core count.
- **The lock-based stack degrades less: 19.5 MOPS at ≥4 threads, 2× the Treiber stack.** Uncontended it is slower (30.7 vs 51.1 MOPS at 1 thread) because each operation pays lock acquire/release on top of the update. Under contention, however, the non-fair `ReentrantLock` lets a running thread re-acquire the lock several times in a row while its cache lines are still local, which effectively batches operations. In the Treiber stack, by contrast, every thread keeps attempting CAS and invalidating the line. The worst case for the lock is exactly 2 threads (8.4 MOPS), where hand-offs between the two threads are most frequent.

- **Read-only (peek) scales up to the core count:** about 2.6× for Treiber and about 2.0× for LockStack at 4 threads, saturating near 1.45–1.56×10^9 ops/s. Read-only access keeps `top` in the Shared state in every core's cache, so no coherence traffic is generated. The sub-linear speedup reflects the per-operation harness cost (workload array, dispatch) and the fact that 1-thread peek is already only about 1.7 ns.
- **N has no effect on the stacks or queues** (within noise), as expected for O(1) operations. N only sets the prefill depth.

6.3 Queue: two-lock vs Michael-Scott

- The same hot-spot behaviour appears. In the update-only workload, MSQueue falls from 38.6 MOPS (1 thread) to 8.2 MOPS (4 threads, speedup 0.21), and LockQueue falls from 29.7 to 17.8 MOPS (0.60).
- **The two-lock queue wins under contention** (about 18 vs 8–10 MOPS update-only, about 33 vs 17 MOPS for 50C-25I-25D). Enqueuers and dequeuers synchronise on different locks and different cache lines, so the two ends genuinely proceed in parallel. In MSQueue, by contrast, every enqueue performs two CASes (link, then swing `tail`), dequeuers also read `tail`, and threads help advance a lagging tail. All of this generates extra CAS failures and shared-memory contention on `head`, `tail` and the last node.
- Uncontended, MSQueue is faster (38.6 vs 29.7 MOPS) because CAS is cheaper than lock + unlock. For read-only peek, LockQueue saturates higher (about 1,410 vs about 1,060 MOPS) because peek on a plain volatile `next` field needs one fewer dependent load than `AtomicReference.get()`.
- The GC-nepotism problem described in Section 2.6 is itself an HPC lesson. Before the fix, memory-management overhead dominated, and young GC pauses of more than 125 ms distorted every queue measurement.

6.4 Treiber vs elimination backoff stack

- **Elimination substantially reduces contention on the central stack.** From 8 to 16 threads, the update-only workload gives 34–41 MOPS for the elimination stack against 9.4–10.3 MOPS for the Treiber stack, i.e. **3.3–4.4× higher throughput**. For 50C-25I-25D, it gives 48–58 MOPS against 18–20 MOPS (about 2.5–3.1×). Results are almost identical for N=1,000 and N=10,000.
- The mechanism: when a CAS on `top` fails, the thread moves to a random exchanger slot instead of retrying immediately. There, a push and a pop cancel out with two or three CASes on a cache line that only those two threads touch. This spreads the traffic across 8 lines instead of 1, and every elimination removes two operations from the central stack.
- The benefit grows with the update fraction. Elimination needs complementary push/pop pairs, so 0C-50I-50D (50/50 push/pop) gains the most, while the read-only workload never fails a CAS and the two stacks are identical (about 1,540 MOPS).
- Elimination does not make the stack *scale*: its speedup is still below 1 (0.63–0.71 at 4–16 threads), because a single core running without contention remains the fastest configuration for an inherently sequential object. What elimination achieves is much more graceful degradation under contention, which is exactly the regime (8–16 threads) the assignment highlights. At 1 thread there is no contention, the backoff path is never taken, and the two stacks perform the same (57 vs 51 MOPS, within run-to-run noise).

6.5 Summary

- **Speedup saturation at p = 4** appears everywhere and is set by the hardware (4 cores). Beyond that, thread scheduling just time-slices the same 4 cores, and throughput stays flat rather than collapsing.
- **Structures with distributed access** (lists) scale close to linearly until saturation. **Structures with a single hot spot** (stacks, queues) show negative speedup under updates, caused by CAS failures and cache-coherence traffic, and only read-only workloads scale.
- **Lock-free is not automatically faster.** It avoids blocking and scales better in the lists, but its higher synchronization overhead per step (atomic wrappers, retries, helping) can make it slower than a well-designed lock-based structure, as the lists, the stacks under contention and the queues all show. Contention-reduction techniques such as elimination (or the two-lock split of the queue) matter more than the choice between locks and CAS.